

## TP2 – Active Object

### Objetivos

Diseñar, Implementar y Usar **Active Object** de **FreeRTOS**

Estructura de datos que representen al dispositivo, incluyendo: **device\_id**, **device\_queue**, etc.

Funciones de Interfaz: **open()**, **release()**, **write()**, **read()**, **ioctl()**, etc.

Patrones de diseño: **Synchronous**, **Asynchronous**, **Latest Input Only**

Gestión del periférico: **Polling**, **Interrupt**, **DMA**

Acceso al periférico: **API STM32-F4 HAL**

Tareas del tipo: **Gatekeeper**

Almacenamiento de datos: **Queue**, **Queue** with **Input and Output Data Spooler** (allocation: **Static**, **Dynamic**)

Usar **Tareas** de **FreeRTOS**

Diseñar, Implementar, Crear, Eliminar **Tareas** (tiempo de procesamiento, secuencia de ejecución, estados, prioridades, instancias, parámetros, cambio de prioridades, procesamiento periódico, IDLE)

Usar **Comunicación** y **Sincronización** de **Tareas** e **Interrupciones** de **FreeRTOS**

Diseñar, Implementar, Comunicar, Sincronizar **Tareas** entre sí y **Tareas** con **Interrupciones**, mediante **Cola** (bloqueantes o no bloqueantes) o **Semáforo binario** y/o **contador** (bloqueantes o no bloqueantes)

Usar **Sección Crítica** de **FreeRTOS**

Diseñar, Implementar, Proteger **Sección Crítica**, mediante **taskENTER\_CRITICAL()/taskEXIT\_CRITICAL()**, **vTaskSuspendAll()/xTaskResumeAll()** y **Semáforo Mutex** (Take/Give)

Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Editar/compilar/depurar programas en **C** con **FreeRTOS**

Usar **Git** y **GitHub**

Generar/actualizar **repositorio** por línea de **comando**

Generar/actualizar **archivos** del tipo **README.md** (**Markdown**)

Usar **Gemini**

Analizar/explicar **código fuente**

### Actividades

[TP2-00 – Formalismos](#)

[TP2-01 – Active Object Led](#)

[TP2-02 – Active Object Sys](#)

[TP2-03 – Active Object Btn](#)

[TP2-04 – Active Object Application](#)

Resolución en grupos de 3 alumnos, requiere **presentación** por parte de un **responsable** del grupo

## TP2 – Actividad 00 – Formalismos

### Objetivos

Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

Usar **Git** y **GitHub**

Generar/Actualizar repositorio

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

Usar **Gemini**

Analizar/Explicar código fuente

**Paso 01:** Crear un nuevo **repositorio público** en **GitHub** para almacenar su **TP2**, pasos:

**GitHub:** Repositories => New =>

Repository name: **sotrii-tp2\_Cohorte-Grupo**

Description: # **CESE - Sistemas Operativos de Tiempo Real II - Trabajo Práctico N°: 2 – Active Object**

Initialize this repository with:

Tildar **Add a README file**

=> **Create repository**

**Campus FIUBA Grado** => **10. SOTR (II) > TP's - Enunciados > Clase 4 > SOTR\_II - Guía TP2 - Object Active**

**Cohorte-Grupo** es: **2XCo202X-01** o **2XCo202X-02** ... etc. (el correspondiente al Curso y Grupo de cursada)

**Paso 02:** Invitar al **repositorio**, como **colaboradores** a los **docentes** de su curso, responsables de la aprobación de su **TP2**, pasos:

**GitHub:** Repositories => Settings => Collaborators => Tildar: **Add people** =>

en **Find people** => tipear: **JuanManuelCruz-FIUBA**

Tildar: **Add JuanManuelCruz-FIUBA**

**GitHub:** Repositories => Settings => Collaborators => Tildar: **Add people** =>

en **Find people** => tipear: **Cpatagon**

Tildar: **Add Cpatagon**

Una vez que los **docentes** de su curso, **hayan aceptado** la **invitación** a ser **colaboradores**, podrá agregarlos como **Revisores**

**PasPaso 03: Asentar** los detalles de las **entregas** en archivos del tipo **README.md** ([Markdown](#), no docx o pdf u otros formatos), **editar** el archivo **README.md** y **modificar** su contenido, pasos:

**GitHub:** **READM.me => Edit this file => borrar** su contenido => **Copiar y Pegar** lo siguiente:

```
# CESE - Sistemas Operativos de Tiempo Real II
## Trabajo Práctico N°: 2 - Object Active
### Cohorte-Grupo: . . . . .
### Modelo de placa: . . . . .

### Responsable de la entrega
| N° SIU | Apellidos, Nombres | Fecha | Deadline |
| :----- | :-----: | :-----: | :-----: |
| XXXXXX | YYYY, ZZ | | Semana 04 |
```

**Actualizar** el archivo **README.md** con los datos de **Cohorte-Grupo**, **Modelo de placa**, del **Responsable de la entrega** (un alumno diferente para cada TP):

**26Co2026 - Grupo\_1**, **NUCLEO-F446RE**, **N° SIU**, **Apellidos** y **Nombres**

## TP2 – Actividad 01 – Active Object Led

### Objetivos

#### Diseñar, Implementar y Usar **Active Object** de **FreeRTOS**

Estructura de datos que representen al dispositivo, incluyendo: **ao\_id**, **ao\_queue**, etc.

Funciones de Interfaz: **open\_ao()**, **release\_ao()**, **send\_event\_ao()**, **ioctl\_ao()**, etc.

Patrones de diseño: **Synchronous**, **Asynchronous**, **Latest Input Only**

Gestión del periférico: **Polling**, **Interrupt**, **DMA**

Acceso al periférico: **API STM32-F4 HAL**

Tareas del tipo: **Gatekeeper**

Almacenamiento de datos: **Queue**, **Queue** with **Input and Output Data Spooler** (allocation: **Static**, **Dynamic**)

#### Usar **Tareas** de **FreeRTOS**

Diseñar, Implementar, Crear, Eliminar **Tareas** (tiempo de procesamiento, secuencia de ejecución, estados, prioridades, instancias, parámetros, cambio de prioridades, procesamiento periódico, IDLE)

#### Usar **Comunicación** y **Sincronización** de **Tareas** e **Interrupciones** de **FreeRTOS**

Diseñar, Implementar, Comunicar, Sincronizar **Tareas** entre sí y **Tareas** con **Interrupciones**, mediante **Cola** (bloqueantes o no bloqueantes) o **Semáforo binario** y/o **contador** (bloqueantes o no bloqueantes)

#### Usar **Sección Crítica** de **FreeRTOS**

Diseñar, Implementar, Proteger **Sección Crítica**, mediante **taskENTER\_CRITICAL()/taskEXIT\_CRITICAL()**, **vTaskSuspendAll()/xTaskResumeAll()** y **Semáforo Mutex** (Take/Give)

#### Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

#### Usar **Git** y **GitHub**

Generar/Actualizar repositorio

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

#### Usar **Gemini**

Analizar/Explicar código fuente

**Paso 01: Generar** un nuevo **proyecto STM32**: **descargar** el archivo [sotrii-tp2\\_01-application.zip](#) e **importar** el **proyecto STM32** que contiene: **sotrii-tp2\_01-application**.

**Compilar** el **proyecto STM32** (**STM32CubeIDE**) importado: **sotrii-tp2\_01-application**.

**Paso 02: Asentar** los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear** el archivo **sotrii-tp2\_01-application.md**.

**Paso 03: Consultar** a **Gemini** de **Google** respecto al código fuente del **proyecto STM32**, pasos:

**Ingresar** a **Gemini** => <https://gemini.google.com/app>, en **Preguntar a Gemini** =>

**Copiar y Pegar** la siguiente línea de **texto**:

**Analizar y explicar** (en español), el funcionamiento del código fuente contenido en los archivos adjuntos: [app.c](#), [task\\_btn.c](#), [task\\_sys.c](#), [task\\_led.c](#), [task\\_btn\\_attribute.c](#), [task\\_sys\\_attribute.h](#) y [task\\_led\\_attribute.h](#).

**Agregar** los siguientes archivos (click en  Agregar archivos => **Subir archivos**):

```
...\sotri_workspace\sotrii-tp2_01-application\app\src\app.c
...\sotri_workspace\sotrii-tp2_01-application\app\src\task_btn.c
...\sotri_workspace\sotrii-tp2_01-application\app\src\task_sys.c
...\sotri_workspace\sotrii-tp2_01-application\app\src\task_led.c
...\sotri_workspace\sotrii-tp2_01-application\app\src\task_btn_attribute.c
...\sotri_workspace\sotrii-tp2_01-application\app\src\task_sys_attribute.c
...\sotri_workspace\sotrii-tp2_01-application\app\src\task_led_attribute.h
```

**Leer y almacenar** la respuesta en: [...\sotri\\_workspace\sotrii-tp2\\_01-application\sotrii-tp2\\_01-application.md](#).

---

**Paso 04: Depurar** el nuevo [proyecto STM32](#).

**Confirmar** mediante la depuración, el análisis, las explicaciones e indicaciones de la respuesta de [Gemini Pro](#) del **Paso anterior**.

---

**Paso 05: Almacenar** el [proyecto STM32: sotrii-tp2\\_01-application](#) y el/los archivo/s de la actividad: [sotrii-tp2\\_01-application.md](#), en el [repositorio](#) creado en [GitHub](#), en el **TP2 – Actividad 01 – Paso 01**.

---

**Paso 06: Diseñar, Implementar y Usar** [Active Object Led](#) con las siguientes características:

Estructura de datos que representen al dispositivo, incluyendo: [ao\\_id](#), [ao\\_queue](#), etc.

Funciones de Interfaz: [open\\_led\\_ao\(\)](#), [release\\_led\\_ao\(\)](#), [send\\_led\\_ao\(\)](#), [ioctl\\_led\\_ao\(\)](#), etc.

Patrón de diseño: [Synchronous](#), [Asynchronous](#), [Latest Input Only](#)

Gestión del periférico: [Polling](#), [Interrupt](#), [DMA](#)

Acceso al periférico: [API STM32-F4 HAL](#)

Una función de active object [Led](#) para instanciar tareas del tipo: [Gatekeeper](#), que reciben como parámetro referencia al active object led que opera.

Almacenamiento de datos: [Queue](#), [Queue with Input and Output Data Spooler](#) (allocation: [Static](#), [Dynamic](#))

---

**Compilar/depurar/observar** comportamiento/**Medir WCET** de las Funciones de Interfaz del driver, **asentar** lo observado en el archivo [sotrii-tp2\\_01-application.md](#) y **registrar** mediante un **video** breve [sotrii-tp2\\_01-application.mp4](#).

---

**Paso 07: Almacenar** el [proyecto STM32: sotrii-tp2\\_01-application](#) y el/los archivo/s de la actividad: [sotrii-tp2\\_01-application.md](#) y [sotrii-tp2\\_01-application.mp4](#), en el [repositorio](#) creado en [GitHub](#), en el **TP2 – Actividad 01 – Paso 01**.

---

## TP2 – Actividad 02 – Active Object Sys

### Objetivos

#### Diseñar, Implementar y Usar **Active Object** de **FreeRTOS**

Estructura de datos que representen al dispositivo, incluyendo: **ao\_id**, **ao\_queue**, etc.

Funciones de Interfaz: **open\_ao()**, **release\_ao()**, **send\_event\_ao()**, **ioctl\_ao()**, etc.

Patrones de diseño: **Synchronous**, **Asynchronous**, **Latest Input Only**

Gestión del periférico: **Polling**, **Interrupt**, **DMA**

Acceso al periférico: **API STM32-F4 HAL**

Tareas del tipo: **Gatekeeper**

Almacenamiento de datos: **Queue**, **Queue** with **Input and Output Data Spooler** (allocation: **Static**, **Dynamic**)

#### Usar **Tareas** de **FreeRTOS**

Diseñar, Implementar, Crear, Eliminar **Tareas** (tiempo de procesamiento, secuencia de ejecución, estados, prioridades, instancias, parámetros, cambio de prioridades, procesamiento periódico, IDLE)

#### Usar **Comunicación** y **Sincronización** de **Tareas** e **Interrupciones** de **FreeRTOS**

Diseñar, Implementar, Comunicar, Sincronizar **Tareas** entre sí y **Tareas** con **Interrupciones**, mediante **Cola** (bloqueantes o no bloqueantes) o **Semáforo binario** y/o **contador** (bloqueantes o no bloqueantes)

#### Usar **Sección Crítica** de **FreeRTOS**

Diseñar, Implementar, Proteger **Sección Crítica**, mediante **taskENTER\_CRITICAL()/taskEXIT\_CRITICAL()**, **vTaskSuspendAll()/xTaskResumeAll()** y **Semáforo Mutex** (Take/Give)

#### Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

#### Usar **Git** y **GitHub**

Generar/Actualizar repositorio

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

#### Usar **Gemini**

Analizar/Explicar código fuente

**Paso 01: Generar** un nuevo **proyecto STM32**: copiar el archivo **sotrii-tp2\_01-application** y pegar/renombrar como **sotrii-tp2\_02-application**.

Renombrar el archivo **sotrii-tp2\_01-application.ioc** como **sotrii-tp2\_02-application.ioc**.

Borrar los archivos de compilación/programación/depuración (**sotrii-tp2\_01-application.launch** y **sotrii-tp2\_01-application.cfg**).

Compilar/Depurar el **proyecto STM32** (**STM32CubeIDE**) renombrado (**sotrii-tp2\_02-application**).

Generar los archivos de depuración (**sotrii-tp2\_02-application.launch** y **sotrii-tp2\_02-application.cfg**).

**Paso 02: Asentar** los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear** el archivo **sotrii-tp2\_02-application.md**.

---

**Paso 03: Diseñar, Implementar y Usar Active Object Sys** de **FreeRTOS**. con las siguientes características:

Estructura de datos que representen al dispositivo, incluyendo: **ao\_id**, **ao\_queue**, etc.

Funciones de Interfaz: **open\_sys\_ao()**, **release\_sys\_ao()**, **send\_sys\_ao()**, **ioctl\_sys\_ao()**, etc.

Patrón de diseño: **Synchronous**, **Asynchronous**, **Latest Input Only**

Gestión del periférico: **Polling**, **Interrupt**, **DMA**

Acceso al periférico: **API STM32-F4 HAL**

Una función de active object **Sys** para instanciar tareas del tipo: **Gatekeeper**, que reciben como parámetro referencia al active object led que opera.

Almacenamiento de datos: **Queue**, **Queue with Input and Output Data Spooler** (allocation: **Static**, **Dynamic**)

**IMPORTANTE:** Active Object **Sys**, debe recibir: “evento y tiempo”

**Compilar/depurar/observar** comportamiento/**Medir WCET** de las Funciones de Interfaz del driver, **asentar** lo observado en el archivo **sotrii-tp2\_02-application.md** y **registrar** mediante un **video** breve **sotrii-tp2\_02-application.mp4**.

---

**Paso 04: Almacenar el proyecto STM32:** **sotrii-tp2\_02-application** y el/los archivo/s de la actividad:

**sotrii-tp2\_02-application.md** y **sotrii-tp2\_02-application.mp4**, en el **repositorio** creado en **GitHub**, en el **TP2 – Actividad 01 – Paso 01**.

---

## TP2 – Actividad 03 – Obejct Active Btn

### Objetivos

#### Diseñar, Implementar y Usar **Active Object** de **FreeRTOS**

Estructura de datos que representen al dispositivo, incluyendo: **ao\_id**, **ao\_queue**, etc.

Funciones de Interfaz: **open\_ao()**, **release\_ao()**, **send\_event\_ao()**, **ioctl\_ao()**, etc.

Patrones de diseño: **Synchronous**, **Asynchronous**, **Latest Input Only**

Gestión del periférico: **Polling**, **Interrupt**, **DMA**

Acceso al periférico: **API STM32-F4 HAL**

Tareas del tipo: **Gatekeeper**

Almacenamiento de datos: **Queue**, **Queue** with **Input and Output Data Spooler** (allocation: **Static**, **Dynamic**)

#### Usar **Tareas** de **FreeRTOS**

Diseñar, Implementar, Crear, Eliminar **Tareas** (tiempo de procesamiento, secuencia de ejecución, estados, prioridades, instancias, parámetros, cambio de prioridades, procesamiento periódico, IDLE)

#### Usar **Comunicación** y **Sincronización** de **Tareas** e **Interrupciones** de **FreeRTOS**

Diseñar, Implementar, Comunicar, Sincronizar **Tareas** entre sí y **Tareas** con **Interrupciones**, mediante **Cola** (bloqueantes o no bloqueantes) o **Semáforo binario** y/o **contador** (bloqueantes o no bloqueantes)

#### Usar **Sección Crítica** de **FreeRTOS**

Diseñar, Implementar, Proteger **Sección Crítica**, mediante **taskENTER\_CRITICAL()/taskEXIT\_CRITICAL()**, **vTaskSuspendAll()/xTaskResumeAll()** y **Semáforo Mutex** (Take/Give)

#### Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

#### Usar **Git** y **GitHub**

Generar/Actualizar repositorio

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

#### Usar **Gemini**

Analizar/Explicar código fuente

**Paso 01: Generar** un nuevo **proyecto STM32**: copiar el archivo **sotrii-tp2\_02-application** y pegar/renombrar como **sotrii-tp2\_03-application**.

**Renombrar** el archivo **sotrii-tp2\_02-application.ioc** como **sotrii-tp2\_03-application.ioc**.

**Borrar** los archivos de compilación/programación/depuración (**sotrii-tp2\_02-application.launch** y **sotrii-tp2\_02-application.cfg**).

**Compilar/Depurar** el **proyecto STM32** (**STM32CubeIDE**) renombrado (**sotrii-tp2\_03-application**).

**Generar** los archivos de depuración (**sotri-tp2\_03-application.launch** y **sotri-tp2\_03-application.cfg**).

**Paso 02: Asentar** los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear** el archivo **sotrii-tp2\_03-application.md**.



---

**Paso 03: Diseñar, Implementar y Usar Active Object Btn de FreeRTOS.** con las siguientes características:

Estructura de datos que representen al dispositivo, incluyendo: **ao\_id**, **ao\_queue**, etc.

Funciones de Interfaz: **open\_btn\_ao()**, **release\_btn\_ao()**, **send\_btn\_ao()**, **ioctl\_btn\_ao()**, etc.

Patrón de diseño: **Synchronous**, **Asynchronous**, **Latest Input Only**

Gestión del periférico: **Polling**, **Interrupt**, **DMA**

Acceso al periférico: **API STM32-F4 HAL**

Una función de active object **Btn** para instanciar tareas del tipo: **Gatekeeper**, que reciben como parámetro referencia al active object led que opera.

Almacenamiento de datos: **Queue**, **Queue with Input and Output Data Spooler** (allocation: **Static**, **Dynamic**)

**IMPORTANTE:** Active Object **Btn**, debe enviar: “evento y tiempo”

**Compilar/depurar/observar** comportamiento/**Medir WCET** de las Funciones de Interfaz del driver, **asentar** lo observado en el archivo **sotrii-tp2\_03-application.md** y **registrar** mediante un **video** breve **sotrii-tp2\_03-application.mp4**.

---

**Paso 04: Almacenar el proyecto STM32: sotrii-tp2\_03-application** y el/los archivo/s de la actividad:

**sotrii-tp2\_03-application.md** y **sotrii-tp2\_03-application.mp4**, en el **repositorio** creado en **GitHub**, en el **TP2 – Actividad 01 – Paso 01**.

---

## TP2 – Actividad 04 – Object Active Application

### Objetivos

#### Diseñar, Implementar y Usar **Active Object** de **FreeRTOS**

Estructura de datos que representen al dispositivo, incluyendo: **ao\_id**, **ao\_queue**, etc.

Funciones de Interfaz: **open\_ao()**, **release\_ao()**, **send\_event\_ao()**, **ioctl\_ao()**, etc.

Patrones de diseño: **Synchronous**, **Asynchronous**, **Latest Input Only**

Gestión del periférico: **Polling**, **Interrupt**, **DMA**

Acceso al periférico: **API STM32-F4 HAL**

Tareas del tipo: **Gatekeeper**

Almacenamiento de datos: **Queue**, **Queue** with **Input and Output Data Spooler** (allocation: **Static**, **Dynamic**)

#### Usar **Tareas** de **FreeRTOS**

Diseñar, Implementar, Crear, Eliminar **Tareas** (tiempo de procesamiento, secuencia de ejecución, estados, prioridades, instancias, parámetros, cambio de prioridades, procesamiento periódico, IDLE)

#### Usar **Comunicación** y **Sincronización** de **Tareas** e **Interrupciones** de **FreeRTOS**

Diseñar, Implementar, Comunicar, Sincronizar **Tareas** entre sí y **Tareas** con **Interrupciones**, mediante **Cola** (bloqueantes o no bloqueantes) o **Semáforo binario** y/o **contador** (bloqueantes o no bloqueantes)

#### Usar **Sección Crítica** de **FreeRTOS**

Diseñar, Implementar, Proteger **Sección Crítica**, mediante **taskENTER\_CRITICAL()/taskEXIT\_CRITICAL()**, **vTaskSuspendAll()/xTaskResumeAll()** y **Semáforo Mutex** (Take/Give)

#### Usar **STM32CubeIDE**

Generar proyecto **STM32** con **FreeRTOS**

Compilar/Depurar programas en **C** con **FreeRTOS**

#### Usar **Git** y **GitHub**

Generar/Actualizar repositorio

Generar/Actualizar archivos del tipo **README.md** (**Markdown**)

#### Usar **Gemini**

Analizar/Explicar código fuente

**Paso 01: Generar** un nuevo **proyecto STM32**: copiar el archivo **sotrii-tp2\_03-application** y pegar/renombrar como **sotrii-tp2\_04-application**.

**Renombrar** el archivo **sotrii-tp2\_03-application.ioc** como **sotrii-tp2\_04-application.ioc**.

**Borrar** los archivos de compilación/programación/depuración (**sotrii-tp2\_03-application.launch** y **sotrii-tp2\_03-application.cfg**).

**Compilar/Depurar** el **proyecto STM32** (**STM32CubeIDE**) renombrado (**sotrii-tp2\_04-application**).

**Generar** los archivos de depuración (**sotri-tp2\_04-application.launch** y **sotri-tp2\_04-application.cfg**).

**Paso 02: Asentar** los detalles de las **entregas** en archivos del tipo **README.md** (**Markdown**, no docx o pdf u otros formatos), **crear** el archivo **sotrii-tp2\_04-application.md**.

**Paso 03: Diseñar, Implementar y Usar [Active Object Btn](#), [Sys](#), Led en [Application](#) de [FreeRTOS](#).** con las siguientes características:

2 (DOS) [AO Btn](#): **BTN\_A** (B1) y **BTN\_B** (D7), envían “evento y tiempo” a:

1 (UN) [AO Sys](#), envía “evento” a:

3 (TRES) [AO Led](#): **LED\_A** (LD2), **LED\_B** (D12) y **LED\_C** (D11)

Normalmente **LED\_A** y **LED\_B** están encendidos; mientras que **LED\_C** está apagado.

Oprimir brevemente **BTN\_A**, hace titilar a **LED\_A** y enciende **LED\_C** por un tiempo de operación **T\_A** = 5 Segundos.  
Oprimir prolongadamente **BTN\_A** modifica el tiempo de operación **T** = “**tiempo**” recibido.

Oprimir brevemente **BTN\_B**, hace titilar a **LED\_B** y enciende **LED\_C** por un tiempo de operación **T\_B** = 10 Segundos.  
Oprimir prolongadamente **BTN\_B** modifica el tiempo de operación **T\_B** = “**tiempo**” recibido.

**Compilar/depurar/observar** comportamiento/**Medir WCET** de las Funciones de Interfaz del driver, **asentar** lo observado en el archivo [sotrii-tp2\\_04-application.md](#) y **registrar** mediante un **video** breve [sotrii-tp2\\_04-application.mp4](#).

**Paso 04: Almacenar el [proyecto STM32: sotrii-tp2\\_04-application](#) y el/los archivo/s de la actividad: [sotrii-tp2\\_04-application.md](#) y [sotrii-tp2\\_04-application.mp4](#), en el [repositorio](#) creado en [GitHub](#), en el TP2 – Actividad 01 – Paso 01.**

**GitHub:** Presionar: **Commit changes ... (Guardar los cambios)**

Tildar: **Create a new branch** for this commit and start a pull request

Presionar: **Propose changes**

Presionar: **Create pull request**, tipear el nombre del **branch** y **agregar** a los [docentes](#) de su curso, como **Reviewers** (para invitarlos a **revisar** sus **entregas** mediante un **pull request**)

Ver **cómo hacer el pull request** en este video: [Trabajo colaborativo con Github](#)

En la descripción del video se detalla el contenido (ver en [14:23](#) la sección “**CÓMO SOLICITAR REVISIONES**”)  
**IMPORTANTE:** Se recomienda que primero vean “[19:25 Solución a un problema muy habitual](#)” y luego “[19:02 Create pull request y Review requested](#)”